

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: CSA16 COURSE CODE: Data Warehousing and Data Mining

COURSE OUTCOME COVERED

CO5: Apply association rule mining techniques to discover interesting relationships and patterns within large datasets
Assessment Tool Weightage: 35%

QUESTIONS: 05

Total Marks:50

Annexure A

Pseudocode Writing Task (Algorithm Design)

Criteria	Problem Understanding (2)	Algorithm Design (3)	Use of Control Structures (2)	Pseudocode Structure (1)	Completeness (2)	Total(10)
Weightage	20%	30%	20%	10%	20%	100%
<i>Q1</i>						
<i>Q2</i>						
<i>Q3</i>						
<i>Q4</i>						
<i>Q5</i>						

Code of Conduct:

*I S. V. K. Sharon Surya(192324246) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

S. Sharon Surya

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Annexure A

Q1. Dataset: Online Shopping Transactions

Order ID	Products Purchased
O1	Mobile, Earphones
O2	Mobile, Charger
O3	Earphones, Charger
O4	Mobile, Earphones, Charger
O5	Mobile, Earphones

Question:

Design pseudocode for the **Apriori algorithm** to discover frequent product combinations from the online shopping dataset using a specified minimum support threshold. Clearly illustrate the candidate itemset generation and pruning process at each iteration. (BL3 – 1.05 Marks)

Q2. Dataset: Library Book Borrowing Records

Borrow ID	Books Borrowed
B1	Data Science, Python
B2	Python, Machine Learning
B3	Data Science, Machine Learning
B4	Data Science, Python, Machine Learning
B5	Python, Machine Learning

Question:

Develop pseudocode for the **FP-Growth algorithm** to identify frequent borrowing patterns without generating candidate itemsets. Explain the steps involved in constructing the FP-Tree using the given borrowing records. (BL4 – 1.05 Marks)

Q3. Dataset: Frequently Purchased Services

Service Combination	Support Count
{Internet}	4
{Streaming}	4
{Cloud Storage}	3

Service Combination	Support Count
{Internet, Streaming}	3
{Streaming, Cloud Storage}	2

Question:

Write pseudocode to generate **association rules** from the given frequent service combinations. Include the procedures for confidence calculation, rule generation, and filtering based on a minimum confidence threshold. *(BL3 – 1.05 Marks)*

Q4. Dataset: Student Course Enrollment

Student ID Courses Enrolled

S1	Python, Database
S2	Python, AI
S3	Database, AI
S4	Python, Database, AI
S5	Python

Question:

Design pseudocode for **constraint-based frequent itemset mining** where only itemsets containing the course "**Python**" are considered during the mining process. Explain how the constraint affects candidate generation and pruning. *(BL4 – 1.05 Marks)*

Q5. Dataset: Hospital Treatment Hierarchy

Patient ID Treatments Received

P1	Healthcare, Cardiology, ECG
P2	Healthcare, Orthopedics
P3	Healthcare, Cardiology
P4	Healthcare, Cardiology, ECG
P5	Healthcare, Orthopedics

Question:

Develop pseudocode for **multilevel association rule mining** using the treatment hierarchy provided in the dataset. Explain how rules are generated at different hierarchy levels and how concept hierarchies improve pattern discovery.

ANSWER SHEET – ANNEXURE A

ANSWER SHEET – ANNEXURE A

CSA16 – Data Warehousing and Data Mining | CO5

Q1. Apriori Algorithm – Online Shopping Transactions

Input: Transactions O1–O5 and a minimum support threshold minsup.

Output: Frequent itemsets and the candidate itemsets generated and pruned at each iteration.

Apriori repeatedly generates candidate k-itemsets from the frequent (k–1)-itemsets. Any candidate is pruned if one of its (k–1)-subsets is not frequent. Support is then counted and only candidates meeting minsup are retained.

For the given data, Mobile has support 4/5, Earphones 3/5 and Charger 3/5. The 2-itemsets have support counts 3, 2 and 2 respectively. The 3-itemset {Mobile, Earphones, Charger} has support count 1.

R PROGRAM

```
# R Program - Q1 Apriori
transactions <- list(
  c("Mobile","Earphones"),
  c("Mobile","Charger"),
  c("Earphones","Charger"),
  c("Mobile","Earphones","Charger"),
  c("Mobile","Earphones")
)
min_support <- 0.40

support_count <- function(itemset, D)
  sum(sapply(D, function(t) all(itemset %in% t)))

items <- sort(unique(unlist(transactions)))
L <- list()

C1 <- lapply(items, function(x) x)
L[[1]] <- Filter(function(x)
  support_count(x, transactions)/length(transactions) >= min_support, C1)

k <- 2
while (length(L[[k-1]]) > 0) {
  prev <- L[[k-1]]
  Ck <- list()

  if (length(prev) >= 2) {
    for (i in 1:(length(prev)-1)) {
      for (j in (i+1):length(prev)) {
        cand <- sort(unique(c(prev[[i]], prev[[j]])))
        if (length(cand) == k)
          Ck[[paste(cand, collapse=",")] ] <- cand
      }
    }
  }

  Ck <- unname(Ck)
  if (length(Ck) == 0) break

  # Apriori pruning
  Ck <- Filter(function(cand) {
    subs <- combn(cand, k-1, simplify=FALSE)
    all(sapply(subs, function(s)
      any(sapply(prev, function(x) setequal(x,s)))))
  }, Ck)
```

```

Lk <- Filter(function(x)
  support_count(x, transactions)/length(transactions) >= min_support, Ck)

if (length(Lk) == 0) break
L[[k]] <- Lk
k <- k + 1
}

```

```

cat("Frequent itemsets:\n")
print(L)

```

Q2. FP-Growth Algorithm – Library Borrowing Records

Input: Borrowing transactions B1–B5 and minsup.

Output: Frequent patterns without explicit candidate generation.

FP-Growth first counts item frequencies, removes infrequent items, orders the remaining items by frequency, and inserts each transaction into a shared FP-Tree. Shared prefixes are compressed by increasing node counts. The tree is mined using conditional pattern bases and conditional FP-Trees.

In the supplied records, Python and Machine Learning each occur 4 times and Data Science occurs 3 times.

R PROGRAM

```

# R Program - Q2 FP-Growth
transactions <- list(
  c("Data Science","Python"),
  c("Python","Machine Learning"),
  c("Data Science","Machine Learning"),
  c("Data Science","Python","Machine Learning"),
  c("Python","Machine Learning")
)
min_support <- 0.40

# Count frequent items
counts <- table(unlist(transactions))
freq <- names(counts[counts/length(transactions) >= min_support])
order_items <- names(sort(counts[freq], decreasing=TRUE))

# FP-tree node
node <- function(item="ROOT", count=0)
  list(item=item, count=count, children=list())

tree <- node()

insert <- function(n, items) {
  if (length(items) == 0) return(n)
  item <- items[1]

  if (item %in% names(n$children)) {
    n$children[[item]]$count <-
      n$children[[item]]$count + 1
  } else {
    n$children[[item]] <- node(item, 1)
  }

  n$children[[item]] <-
    insert(n$children[[item]], items[-1])
  n
}

for (t in transactions) {
  t <- t[t %in% freq]
  t <- order_items[order_items %in% t]
  tree <- insert(tree, t)
}

```

```

print_tree <- function(n, level=0) {
  if (length(n$children) == 0) return()
  for (x in n$children) {
    cat(paste(rep(" ", level), collapse=""),
        x$item, ":", x$count, "\n")
    print_tree(x, level+1)
  }
}

cat("FP-Tree:\n")
print_tree(tree)
cat("The tree is mined using conditional pattern bases and conditional FP-trees.\n")

```

Q3. Association Rule Generation – Frequently Purchased Services

For a rule $A \rightarrow B$:

$$\text{Confidence}(A \rightarrow B) = \text{Support}(A \cup B) / \text{Support}(A).$$

From the supplied counts:

$$\text{Internet} \rightarrow \text{Streaming} = 3/4 = 75\%.$$

$$\text{Streaming} \rightarrow \text{Internet} = 3/4 = 75\%.$$

$$\text{Streaming} \rightarrow \text{Cloud Storage} = 2/4 = 50\%.$$

$$\text{Cloud Storage} \rightarrow \text{Streaming} = 2/3 \approx 66.67\%.$$

The R program below filters the rules using a minimum confidence of 60%.

R PROGRAM

```

# R Program - Q3 Association Rules
support <- c(
  "Internet"=4,
  "Streaming"=4,
  "Cloud Storage"=3,
  "Internet,Streaming"=3,
  "Streaming,Cloud Storage"=2
)

min_confidence <- 0.60

rules <- data.frame(
  Rule=c(
    "Internet -> Streaming",
    "Streaming -> Internet",
    "Streaming -> Cloud Storage",
    "Cloud Storage -> Streaming"
  ),
  Confidence=c(
    support["Internet,Streaming"]/support["Internet"],
    support["Internet,Streaming"]/support["Streaming"],
    support["Streaming,Cloud Storage"]/support["Streaming"],
    support["Streaming,Cloud Storage"]/support["Cloud Storage"]
  )
)

rules$ConfidencePercent <- rules$Confidence * 100
rules$Accepted <- rules$Confidence >= min_confidence

```

```

print(rules)
cat("\nRules satisfying minimum confidence:\n")
print(subset(rules, Accepted))

```

Q4. Constraint-Based Frequent Itemset Mining – Student Course Enrollment

Constraint: only itemsets containing Python are considered.

Python occurs in S1, S2, S4 and S5, giving support $4/5 = 80\%$. During candidate generation, any itemset that does not contain Python is discarded immediately. This reduces the search space and makes mining more efficient.

R PROGRAM

```
# R Program - Q4 Constraint-Based Mining
transactions <- list(
  c("Python", "Database"),
  c("Python", "AI"),
  c("Database", "AI"),
  c("Python", "Database", "AI"),
  c("Python")
)
min_support <- 0.40
required_item <- "Python"

support_count <- function(itemset, D)
  sum(sapply(D, function(t) all(itemset %in% t)))

items <- sort(unique(unlist(transactions)))
all_candidates <- list()

for (k in 1:length(items)) {
  combos <- combn(items, k, simplify=FALSE)

  # Constraint is applied during candidate generation
  valid <- Filter(function(x) required_item %in% x, combos)
  all_candidates <- c(all_candidates, valid)
}

frequent <- Filter(function(x)
  support_count(x, transactions)/length(transactions) >= min_support,
  all_candidates)

cat("Frequent itemsets containing Python:\n")
for (x in frequent) {
  cat("{", paste(x, collapse=" "), "}",
      " Support = ",
      support_count(x, transactions)/length(transactions),
      "\n", sep="")
}
```

Q5. Multilevel Association Rule Mining – Hospital Treatment Hierarchy

The supplied hierarchy is considered from general to specific concepts. Healthcare occurs in all five records, Cardiology occurs in three, Orthopedics in two, and ECG in two.

Mining at higher levels identifies broad treatment patterns; mining at lower levels identifies more specific relationships. The R program demonstrates support calculation at the different levels and an example confidence calculation for Cardiology → ECG.

R PROGRAM

```
# R Program - Q5 Multilevel Association Rule Mining
transactions <- list(
  c("Healthcare", "Cardiology", "ECG"),
  c("Healthcare", "Orthopedics"),
  c("Healthcare", "Cardiology"),
  c("Healthcare", "Cardiology", "ECG"),
  c("Healthcare", "Orthopedics")
)

support <- function(itemset)
  mean(sapply(transactions, function(t) all(itemset %in% t)))

min_support <- 0.40
min_confidence <- 0.60

# Level 1: general concept
level1 <- list(c("Healthcare"))

# Level 2: specific concepts
level2 <- list(c("Cardiology"), c("Orthopedics"))

# Level 3: more specific concept
```



```

level3 <- list(c("Cardiology","ECG"))

cat("LEVEL 1\n")
for (x in level1)
  cat("{",paste(x,collapse=" ")," } Support = ",support(x),"n",sep="")

cat("\nLEVEL 2\n")
for (x in level2)
  cat("{",paste(x,collapse=" ")," } Support = ",support(x),"n",sep="")

cat("\nLEVEL 3\n")
for (x in level3)
  cat("{",paste(x,collapse=" ")," } Support = ",support(x),"n",sep="")

# Example detailed association rule
conf <- support(c("Cardiology","ECG")) / support(c("Cardiology"))
cat("\nRule: Cardiology -> ECG\n")
cat("Confidence =", round(conf*100,2), "%\n")

if (conf >= min_confidence)
  cat("Rule accepted\n")
else
  cat("Rule rejected\n")

```

Note: The original assessment document is preserved first, followed by each question's answer and its R program directly underneath.